

TP1 — Manipulation et intégration de données

Nassim EL HADDAD

Table des matières

1	Étape 1 — Construction du jeu de données	2
1.1	Question 1 — Étude des données, chargement et cohérence	2
1.2	Question 2 — Rétroconception et MCD	4
1.3	Question 3 — Schéma SQL et exécution	5
1.4	Question 4 — Règles de correspondance et de chargement	6
1.5	Question 5 — Chargement et vérification	6
1.6	Question 6 — Contraintes d'intégrité	8
2	Étape 2 — Exploitation du jeu de données	9
2.1	Dix requêtes SQL statistiques	9
2.2	Analyse après intégration (ydata_profiling)	12
3	Conclusion	12
A	Annexe — Rapport ydata_profiling	13

1 Étape 1 — Construction du jeu de données

1.1 Question 1 — Étude des données, chargement et cohérence

Pour commencer, les cinq fichiers fournis ont été examinés puis chargés dans une base SQLite temporaire, afin de pouvoir les parcourir et tester leur cohérence avant toute modélisation.

Fichier	Lignes	Séparateur	Encodage	En-tête
CIS_bdpm.txt	15 883	tabulation	latin-1	non
CIS_CIP_bdpm.txt	20 925	tabulation	utf-8	non
CIS_COMPO_bdpm.txt	32 439	tabulation	latin-1	non
CIS_GENER_bdpm.txt	10 737	tabulation	latin-1	non
healthcare_data.csv	17 498	virgule	utf-8	oui

Plusieurs écarts apparaissent par rapport au sujet. Le CSV n'est pas séparé par des points-virgules mais par des virgules, et les fichiers BDPM n'ont pas de ligne d'en-tête. On trouve aussi des attributs multivalués (titulaires, voies d'administration), des redondances dans COMPO et GENER, ainsi que des prix au format français du type 24,34.

Les captures ci-dessous illustrent l'étude Python des fichiers (dimensions, aperçus, particularités), puis le chargement SQLite et les contrôles de cohérence.

```
1. ETUDE DES DONNEES FOURNIES
... CIS_bdpm.txt ...
Lignes : 15,883 | Colonnes : 12
Colonnes : ['code_cis', 'denomination', 'forme', 'voies_admin', 'statut_admin', 'type_procedure', 'etat_commercialisation', 'date_ama', 'statut_bdp', 'nature_autorisation', 'titulaires', 'surveillance_renforcement']
Champs vides (row) :
statut_bdp : 13652
nature_autorisation : 13558
... CIS_CIP_bdpm.txt ...
Lignes : 20,925 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... CIS_COMPO_bdpm.txt ...
Lignes : 32,439 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... CIS_GENER_bdpm.txt ...
Lignes : 10,737 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... healthcare_data.csv ...
Lignes : 17,498 | Colonnes : 13
Colonnes : ['patient_id', 'first_name', 'last_name', 'age', 'gender', 'phone', 'email', 'department', 'diagnosis', 'treatment', 'admission_date', 'discharge_date', 'status', 'bill_amount']
Champs vides (row) :
patient_id : 13652
first_name : 13558
last_name : 13558
age : 13652
gender : 13558
phone : 13558
email : 13558
department : 13558
diagnosis : 13558
treatment : 13558
admission_date : 13558
discharge_date : 13558
status : 13558
bill_amount : 13558
```

(a) CIS_bdpm et CIS_CIP

```
... CIS_COMPO_bdpm.txt ...
Lignes : 32,439 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... CIS_GENER_bdpm.txt ...
Lignes : 10,737 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... healthcare_data.csv ...
Lignes : 17,498 | Colonnes : 13
Colonnes : ['patient_id', 'first_name', 'last_name', 'age', 'gender', 'phone', 'email', 'department', 'diagnosis', 'treatment', 'admission_date', 'discharge_date', 'status', 'bill_amount']
Champs vides (row) :
patient_id : 13652
first_name : 13558
last_name : 13558
age : 13652
gender : 13558
phone : 13558
email : 13558
department : 13558
diagnosis : 13558
treatment : 13558
admission_date : 13558
discharge_date : 13558
status : 13558
bill_amount : 13558
```

(b) COMPO, GENER et healthcare

Figure 1 – Étude des fichiers sources (Python).

```
2. CHARGEMENT DANS SQLITE
... CIS_bdpm.txt ...
Lignes : 15,883 | Colonnes : 12
Colonnes : ['code_cis', 'denomination', 'forme', 'voies_admin', 'statut_admin', 'type_procedure', 'etat_commercialisation', 'date_ama', 'statut_bdp', 'nature_autorisation', 'titulaires', 'surveillance_renforcement']
Champs vides (row) :
statut_bdp : 13652
nature_autorisation : 13558
... CIS_CIP_bdpm.txt ...
Lignes : 20,925 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... CIS_COMPO_bdpm.txt ...
Lignes : 32,439 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... CIS_GENER_bdpm.txt ...
Lignes : 10,737 | Colonnes : 13
Colonnes : ['code_cis', 'code_cip', 'libelle', 'statut_admin', 'etat_commercialisation', 'date_declaration', 'code_cip1', 'agrement_collectif', 'taux_remboursement', 'prix_public', 'horaires', 'indication_remboursement', 'prix_medicament', 'prix_public']
Champs vides (row) :
indication_remboursement : 20227
horaires : 2224
taux_remboursement : 2222
prix_medicament : 2222
prix_public : 2222
... healthcare_data.csv ...
Lignes : 17,498 | Colonnes : 13
Colonnes : ['patient_id', 'first_name', 'last_name', 'age', 'gender', 'phone', 'email', 'department', 'diagnosis', 'treatment', 'admission_date', 'discharge_date', 'status', 'bill_amount']
Champs vides (row) :
patient_id : 13652
first_name : 13558
last_name : 13558
age : 13652
gender : 13558
phone : 13558
email : 13558
department : 13558
diagnosis : 13558
treatment : 13558
admission_date : 13558
discharge_date : 13558
status : 13558
bill_amount : 13558
```

(a) Chargement SQLite et cohérence

```
> Nombre de cis distincts par table
table_name  nb_cis
cis_bdpm    15883
cis_cip      20925
cis_compo    32439
cis_gener     10737
cis_healthcare 17498

> Doublets potentiels code_cip dans cis_bdpm
Empty DataFrame
Columns: [code_cip, nb]
Index: []

> Doublets potentiels code_cip dans cis_cip_bdpm
Empty DataFrame
Columns: [code_cip, nb]
Index: []

> Doublets potentiels code_cip dans cis_compo
Empty DataFrame
Columns: [code_cip, nb]
Index: []

> Doublets potentiels code_cip dans cis_gener
Empty DataFrame
Columns: [code_cip, nb]
Index: []

> Doublets potentiels code_cip dans cis_healthcare
Empty DataFrame
Columns: [code_cip, nb]
Index: []
```

(b) CIS distincts et absence de doublons

Figure 2 – Chargement initial et vérifications SQL.

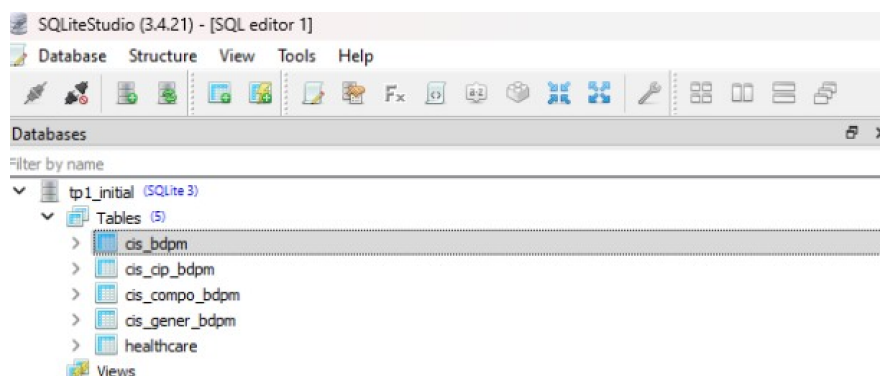


Figure 3 – Les cinq tables sources une fois chargées dans SQLite.

Les requêtes SQL de cohérence confirment que les sources ne sont pas toujours parfaitement reliées entre elles :

- 1 249 médicaments sans présentation ;
- 4 présentations orphelines (CIS absent) ;
- 2 médicaments sans composition ;
- 7 692 médicaments hors groupe générique ;
- 2 483 liens GENER orphelins ;
- aucun doublon sur CIS / CIP.

Ces anomalies guident la suite du travail : lors du chargement dans le schéma cible, les lignes dont la clé étrangère est absente sont filtrées.

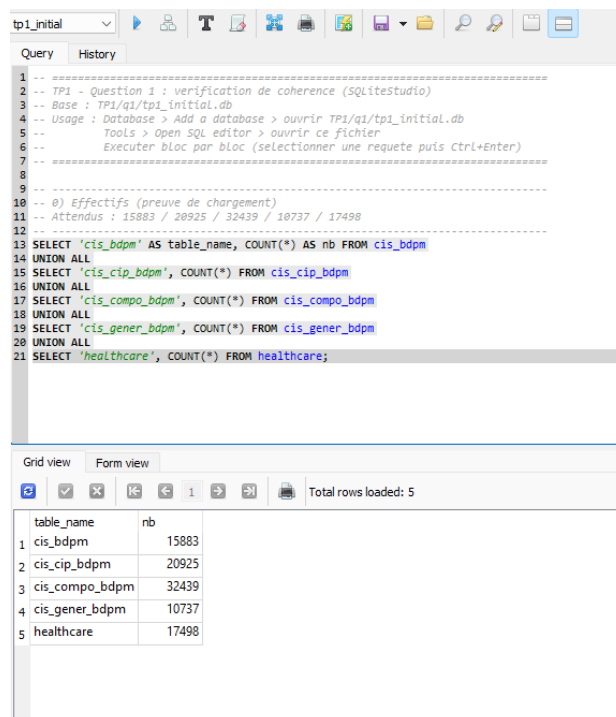


Figure 4 – Vérification des effectifs après le chargement initial.

```

1
2 -- 1) Médicaments SANS aucune présentation (CIP)
3 -- Attendu : 1249
4
5 SELECT COUNT(*) AS nb_medicaments_sans_presentation
6 FROM cis_bdp m
7 LEFT JOIN (SELECT DISTINCT code_cis FROM cis_cip_bdp) p
8 ON m.code_cis = p.code_cis
9 WHERE p.code_cis IS NULL;

```

nb_medicaments_sans_presentation
1249

Figure 5 – Médicaments sans présentation CIP.

```

1
2 -- 2) Présentations dont le CIS est absent de cis_bdp (orphelins)
3 -- Attendu : 4
4
5 SELECT COUNT(*) AS nb_cip_orphelins
6 FROM cis_cip_bdp p
7 LEFT JOIN cis_bdp m ON p.code_cis = m.code_cis
8 WHERE m.code_cis IS NULL;
9
10
11 -- Détail des CIP orphelins (optionnel)
12 SELECT p.code_cis, p.code_cip7, p.libelle
13 FROM cis_cip_bdp p
14 LEFT JOIN cis_bdp m ON p.code_cis = m.code_cis
15 WHERE m.code_cis IS NULL;

```

code_cis	code_cip7	libelle
63278664	5827046	100 sachet(s)-dose(s) papier aluminium polyéthylène de 153,45 mg
64917175	3591088	12 poche(s) PVC surembalee(s) de 500 ml avec embout trocable DAT
65999410	3010515	flacon(s) polyéthylène haute densité (PEHD) avec fermeture de sécurité enfant de 30 comprimé(s)
69912584	4900402	plaque(s) PVC-Aluminium de 7 gélule(s)

Figure 6 – Présentations orphelines (CIS absent).

1.2 Question 2 — Rétroconception et MCD

Le schéma cible fourni par le sujet est volontairement incomplet. Il décrit Patients, Médicaments, Présentations, Substances et Groupes, mais il faut encore introduire les laboratoires et, surtout, l'association entre patients et médicaments, qui n'existe dans aucun fichier source.

Le MCD conserve donc les entités du sujet, ajoute Laboratoire, et pose les associations suivantes :

- **PRENDRE** (Patient–Médicament) : datePrise, posologie, durée ; cardinalités (1,n)–(0,n) ;
- **DISPOSER** (Médicament–Présentation) : (0,n)–(1,1) ;
- **COMPOSER** (Médicament–Substance) : dosage, refDosage ;
- **APPARTENIR** (Médicament–Groupe) : typeGenerique, numeroTri ;
- **DETENIR** (Laboratoire–Médicament) : titulaires AMM.

Côté attributs, Substances reprend **denomination**, **dosage** et **refDosage** comme demandé. Comme une même substance peut avoir plusieurs dosages selon le médicament, ces informations sont également portées par COMPOSER, pour ne pas perdre le détail issu de CIS_COMPO.

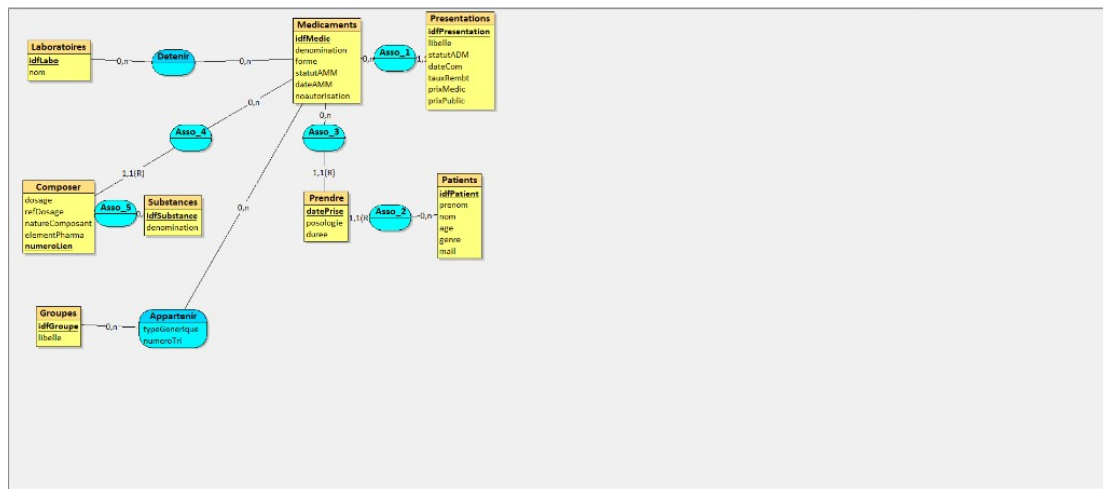


Figure 7 – Modèle conceptuel réalisé avec Looping-MCD.

Le schéma relationnel déduit est le suivant (clés soulignées) :

- Patients(idfPatient, prenom, nom, age, genre, mail);
- Médicaments(idfMedic, denomination, forme, statutAMM, dateAMM, noautorisation);
- Présentations(idfPresentation, ..., idfMedic) avec **idfPresentation = code CIP7**;
- Substances(idfSubstance, denomination, dosage, refDosage);
- Groupes(idfGroupe, libelle);
- Laboratoires(idfLabo, nom);
- Prendre, Composer, Appartenir, Detenir (tables d'association).

1.3 Question 3 — Schéma SQL et exécution

Le MCD est ensuite traduit en SQL SQLite. Les associations 1,N deviennent des clés étrangères, tandis que les associations N,N donnent naissance à des tables d'association. Après exécution du script, les dix tables existent bien, encore vides — le chargement intervient à la question 5.

```
PRAGMA foreign_keys = ON;

CREATE TABLE Medicaments (
  idfMedic INTEGER PRIMARY KEY, denomination TEXT NOT NULL,
  forme TEXT, statutAMM TEXT, dateAMM TEXT, noautorisation TEXT);
CREATE TABLE Patients (
  idfPatient TEXT PRIMARY KEY, prenom TEXT, nom TEXT,
  age INTEGER, genre TEXT, mail TEXT);
CREATE TABLE Presentations (
  idfPresentation TEXT PRIMARY KEY, -- code CIP7
  libelle TEXT, statutADM TEXT, dateCom TEXT, tauxRembt TEXT,
  prixMedic REAL, prixPublic REAL, idfMedic INTEGER NOT NULL,
  FOREIGN KEY (idfMedic) REFERENCES Medicaments(idfMedic));
CREATE TABLE Substances (
  idfSubstance TEXT PRIMARY KEY, denomination TEXT NOT NULL,
  dosage TEXT, refDosage TEXT);
CREATE TABLE Groupes (
  idfGroupe INTEGER PRIMARY KEY, libelle TEXT NOT NULL);
CREATE TABLE Laboratoires (
  idfLabo INTEGER PRIMARY KEY, nom TEXT NOT NULL);
CREATE TABLE Prendre (
  idfPatient TEXT NOT NULL, idfMedic INTEGER NOT NULL,
  datePrise TEXT NOT NULL, posologie INTEGER NOT NULL, duree INTEGER NOT NULL,
  PRIMARY KEY (idfPatient, idfMedic, datePrise),
  FOREIGN KEY (idfPatient) REFERENCES Patients(idfPatient),
  FOREIGN KEY (idfMedic) REFERENCES Medicaments(idfMedic));
CREATE TABLE Composer (
  idfMedic INTEGER NOT NULL, idfSubstance TEXT NOT NULL,
  dosage TEXT, refDosage TEXT, natureComposant TEXT,
  elementPharma TEXT, numeroLien TEXT NOT NULL DEFAULT '1',
  PRIMARY KEY (idfMedic, idfSubstance, numeroLien),
  FOREIGN KEY (idfMedic) REFERENCES Medicaments(idfMedic),
  FOREIGN KEY (idfSubstance) REFERENCES Substances(idfSubstance));
CREATE TABLE Appartenir (
  idfMedic INTEGER NOT NULL, idfGroupe INTEGER NOT NULL,
  typeGenerique INTEGER, numeroTri INTEGER,
  PRIMARY KEY (idfMedic, idfGroupe),
  FOREIGN KEY (idfMedic) REFERENCES Medicaments(idfMedic),
  FOREIGN KEY (idfGroupe) REFERENCES Groupes(idfGroupe));
CREATE TABLE Detenir (
  idfLabo INTEGER NOT NULL, idfMedic INTEGER NOT NULL,
  PRIMARY KEY (idfLabo, idfMedic),
  FOREIGN KEY (idfLabo) REFERENCES Laboratoires(idfLabo),
  FOREIGN KEY (idfMedic) REFERENCES Medicaments(idfMedic));
```

1.4 Question 4 — Règles de correspondance et de chargement

Source	Cible	Codage
CIS_bdpm	Médicaments (code_cis → idfMedic, ...)	Python/SQL*
titulaires	Laboratoires(nom) + Detenir (split « ; »)	Python
CIS_CIP	Présentations (CIP7, prix 24,34→24.34)	Python
CIS_COMPO	Substances (DISTINCT) + Composer	Python/SQL*
CIS_GENER	Groupes (DISTINCT) + Appartenir	Python/SQL*
healthcare	Patients	Python
—	Prendre (1–10 médicaments / patient)	Python

*SQL possible après staging (INSERT...SELECT, DISTINCT).

L'association Prendre n'apparaît dans aucun fichier : elle est donc générée. Pour chaque patient, on tire entre 1 et 10 médicaments, une date de prise entre le 1^{er} janvier 2025 et aujourd'hui, une posologie entre 1 et 5, et une durée entre 1 et 30 jours.

L'ordre de chargement respecte les clés étrangères : d'abord les entités, puis les associations, et enfin Prendre. Une partie du travail peut se faire en SQL après un import brut ; le reste (encodages, titulaires multivalués, conversion des prix, génération aléatoire) nécessite Python.

```
SEED = 42
rng = random.Random(SEED)
start, end = date(2025, 1, 1), date.today()
span = max(0, (end - start).days)
for pid in patients:
    k = rng.randint(1, 10)
    for mid in rng.sample(medic_list, k=min(k, len(medic_list))):
        d = start + timedelta(days=rng.randint(0, span))
        rows.append((pid, mid, d.isoformat(),
                     rng.randint(1, 5), rng.randint(1, 30)))
# prix : float(valeur.replace(",", "."))
```

1.5 Question 5 — Chargement et vérification

Le script Python charge ensuite les données dans le schéma cible (graine aléatoire 42, pour la reproductibilité). Les volumes obtenus sont cohérents avec les fichiers sources, une fois les orphelins écartés.

Relation	COUNT(*)	Relation	COUNT(*)
Médicaments	15 883	Laboratoires	666
Patients	17 498	Prendre	96 834
Présentations	20 921	Composer	32 422
Substances	3 907	Appartenir	8 254
Groupes	1 686	Detenir	15 883

Les contrôles d'intégrité sont satisfaisants : aucune présentation orpheline, aucun lien Prendre invalide, et chaque patient dispose d'au moins un médicament. La distribution de Prendre respecte bien l'intervalle 1–10 demandé (moyenne ≈ 5,5).

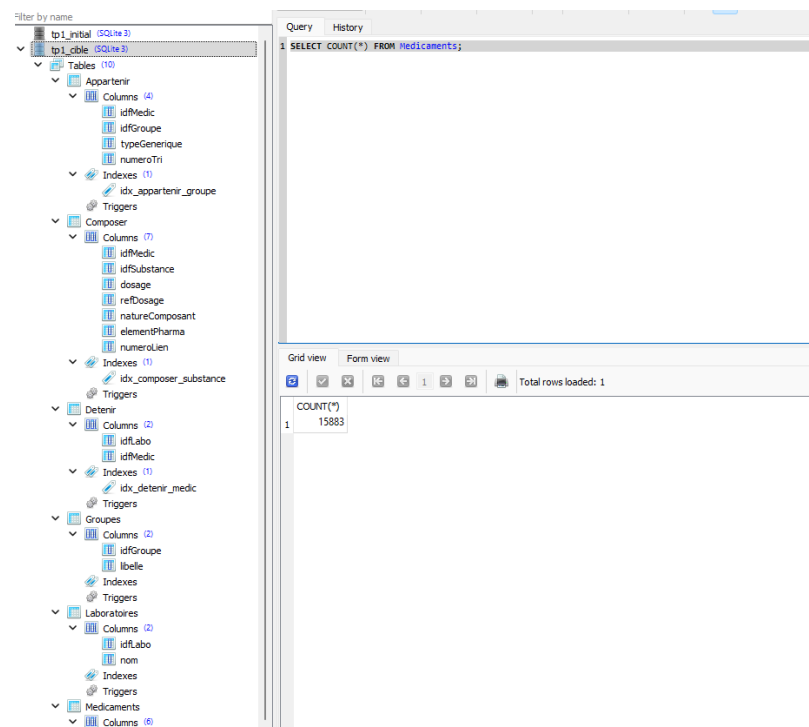


Figure 8 – Preuve de chargement : 15 883 médicaments dans la base cible.

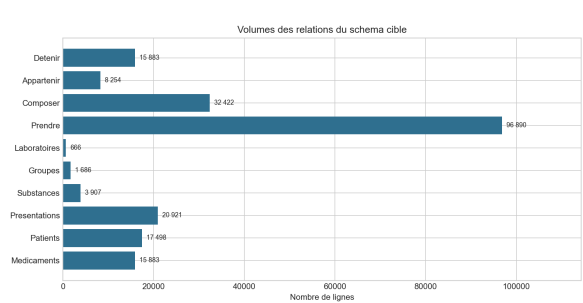


Figure 9 – Nombre de lignes par table après chargement.

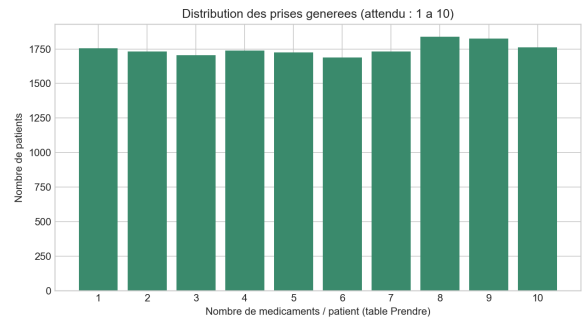


Figure 10 – Distribution du nombre de médicaments par patient.

1.6 Question 6 — Contraintes d'intégrité

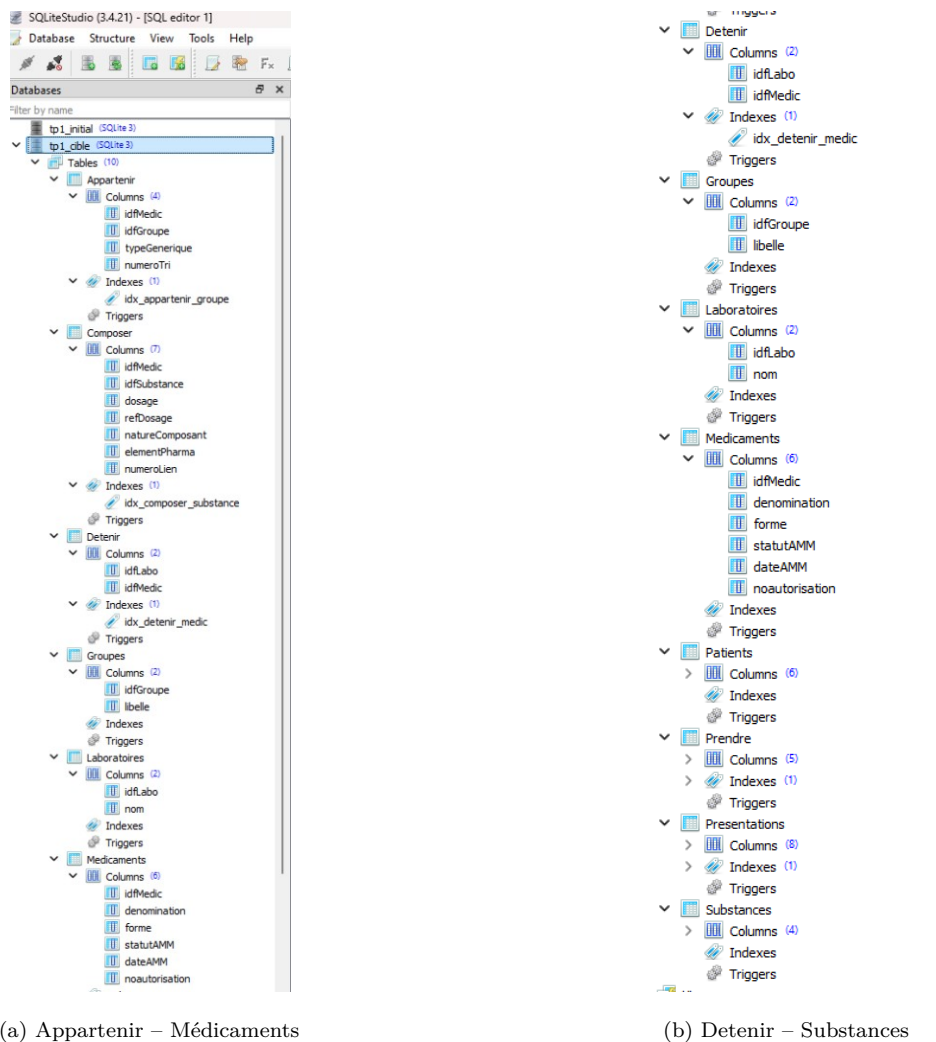


Figure 11 – Les dix tables du schéma cible dans SQLiteStudio.

Le schéma est enfin consolidé avec les clés primaires, les clés étrangères et des contraintes de domaine. Sous SQLite, les CHECK se définissent à la création des tables : le script recrée donc le schéma enrichi, puis remigre les données. Principales contraintes ajoutées :

```
CHECK (age IS NULL OR (age >= 0 AND age <= 120))
CHECK (genre IS NULL OR genre IN ('Male', 'Female', 'Other'))
CHECK (prixMedic IS NULL OR prixMedic >= 0)
CHECK (posologie BETWEEN 1 AND 5)
CHECK (duree BETWEEN 1 AND 30)
CHECK (datePrise >= '2025-01-01')
CHECK (typeGenerique IS NULL OR typeGenerique IN (0, 1, 2, 4))
```

Avec `PRAGMA foreign_keys = ON`, les insertions volontairement incorrectes sont toutes refusées (CHECK ou FOREIGN KEY).

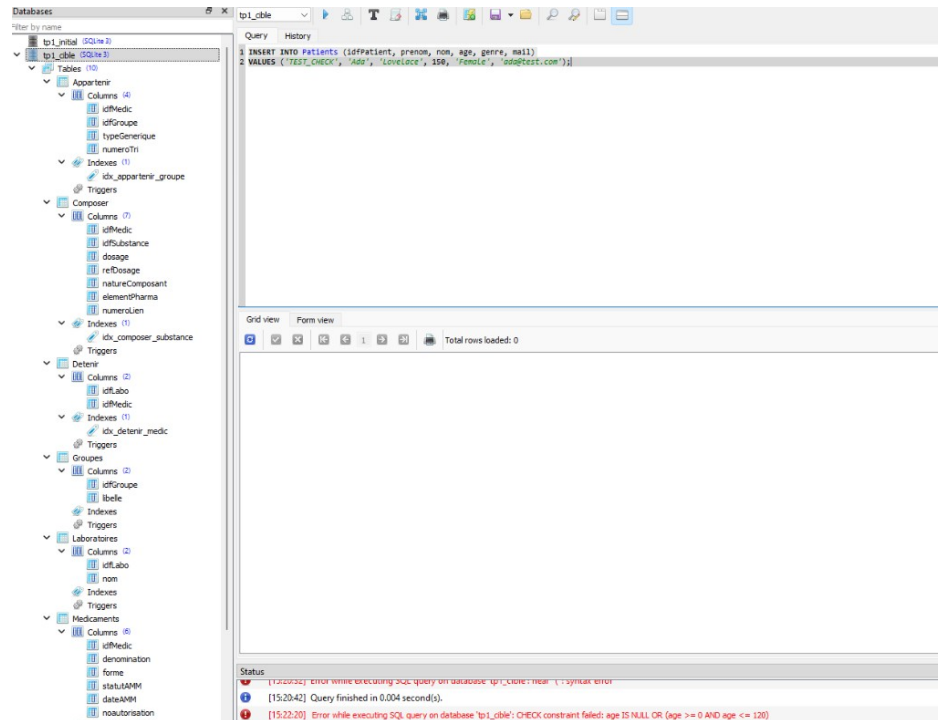


Figure 12 – Exemple de contrainte CHECK : refus d'un patient âgé de 150 ans.

2 Étape 2 — Exploitation du jeu de données

2.1 Dix requêtes SQL statistiques

Dix requêtes statistiques ont été écrites, dont neuf avec **GROUP BY** (le sujet en demande au moins six). Pour chacune, le SQL et un extrait du résultat sont présentés ci-dessous.

Q01 — Présentations par médicament

```
SELECT m.denomination, COUNT(p.idfPresentation) AS nb
FROM Medicaments m
LEFT JOIN Presentations p ON p.idfMedic = m.idfMedic
GROUP BY m.idfMedic, m.denomination
ORDER BY nb DESC LIMIT 15;
```

Dénomination (extrait)	nb
ZOLGENSMA ...	37
CHLORURE DE SODIUM 0,9% LAVOISIER ...	34
CHLORURE DE SODIUM 0,9% AGUETTANT ...	28

Q02 — Médicaments par laboratoire

```
SELECT l.nom, COUNT(d.idfMedic) AS nb
FROM Laboratoires l JOIN Detenir d ON d.idfLabo = l.idfLabo
GROUP BY l.idfLabo, l.nom ORDER BY nb DESC LIMIT 15;
```

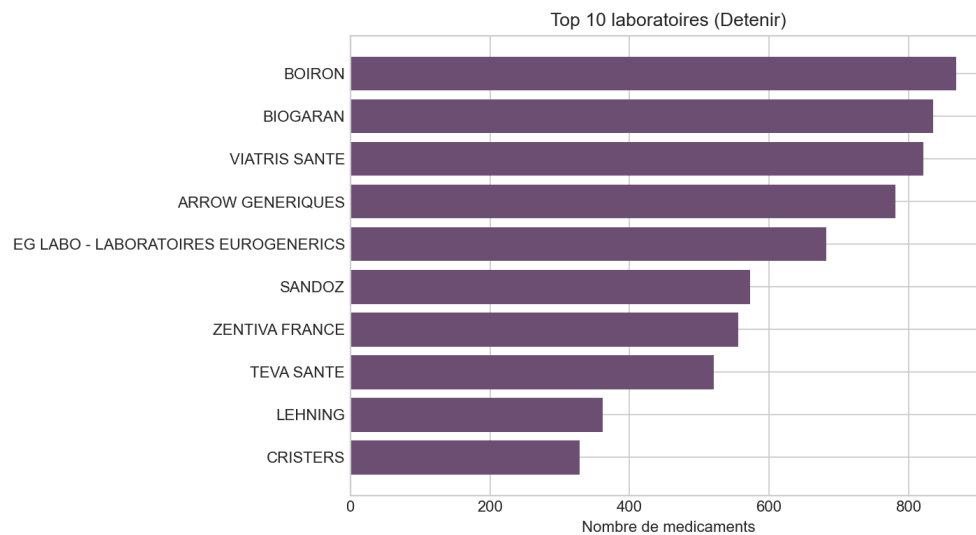
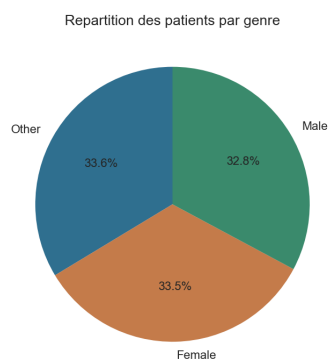


Figure 13 – Laboratoires ayant le plus de médicaments.

Laboratoire	nb
BOIRON	868
BIOGARAN	835
VIATRIS SANTE	821

Q03 — Patients par genre

```
SELECT genre, COUNT(*), ROUND(AVG(age),1), MIN(age), MAX(age)
FROM Patients GROUP BY genre ORDER BY COUNT(*) DESC;
```



Genre	nb	moy.	min	max
Other	5886	45.8	1	90
Female	5870	45.2	1	90
Male	5742	45.3	1	90

Figure 14 – Répartition par genre.

Q04 — Distribution des prises

```
SELECT nb_prises, COUNT(*) AS nb_patients FROM (
  SELECT idfPatient, COUNT(*) AS nb_prises FROM Prendre GROUP BY idfPatient
) GROUP BY nb_prises ORDER BY nb_prises;
```

On obtient une distribution quasi uniforme : environ 1 750 patients pour chaque valeur de 1 à 10, ce qui est cohérent avec un tirage aléatoire.

Q05 — Top substances

```
SELECT s.denomination, COUNT(DISTINCT c.idfMedic) AS nb
FROM Substances s JOIN Composer c ON c.idfSubstance = s.idfSubstance
GROUP BY s.idfSubstance, s.denomination ORDER BY nb DESC LIMIT 15;
```

Substance	nb médicaments
HYDROCHLOROTHIAZIDE	280
PARACTAMOL	221
AMLODIPINE	166

Q06 — Types génériques

```
SELECT typeGenerique, COUNT(*), COUNT(DISTINCT idfMedic)
FROM Appartenir GROUP BY typeGenerique ORDER BY typeGenerique;
```

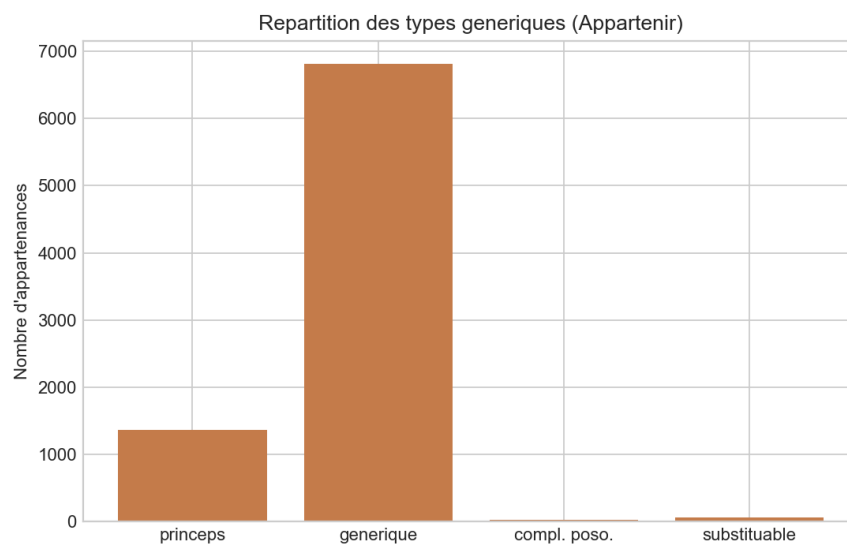


Figure 15 – Types génériques (Appartenir).

Type	appartenances	médicaments
0 (princeps)	1360	1358
1 (générique)	6819	6775
2 / 4	23 / 52	23 / 52

Q07 — Prix moyen par taux de remboursement

```
SELECT COALESCE(NULLIF(tauxRembt,''), '(vide)') AS taux,
COUNT(*) AS nb, ROUND(AVG(prixMedic),2) AS prix_moy
FROM Presentations GROUP BY tauxRembt ORDER BY nb DESC LIMIT 15;
```

On observe deux écritures du même taux (« 65% » et « 65 % »), ainsi qu'une forte proportion de prix manquants (environ 37 % des présentations).

Q08 — Volumes globaux

```
SELECT (SELECT COUNT(*) FROM Medicaments),
(SELECT COUNT(*) FROM Presentations),
(SELECT COUNT(*) FROM Patients),
(SELECT COUNT(*) FROM Prendre),
(SELECT ROUND(AVG(nb),2) FROM (
SELECT COUNT(*) AS nb FROM Prendre GROUP BY idfPatient));
```

Au total : 15 883 médicaments, 20 921 présentations, 17 498 patients et 96 834 prises, soit 5,53 prises par patient en moyenne.

Q09 — Top médicaments via Prendre

```
SELECT m.denomination, COUNT(*) AS nb_prises
FROM Prendre pr JOIN Medicaments m ON m.idfMedic = pr.idfMedic
GROUP BY m.idfMedic, m.denomination ORDER BY nb_prises DESC LIMIT 15;
```

Ce classement n'a pas de sens clinique : l'association Prendre a été générée aléatoirement.

Q10 — Prises par tranche d'âge

```
SELECT tranche_age, COUNT(DISTINCT idfPatient),
       ROUND(1.0*COUNT(*)/COUNT(DISTINCT idfPatient),2)
FROM (...) GROUP BY tranche_age;
```

Tranche	patients	prises/patient
0-17	3331	5.55
18-39	4262	5.46
40-64	4784	5.53
65+	5121	5.59

L'intensité des prises reste comparable d'une tranche d'âge à l'autre : la génération n'a pas favorisé une classe en particulier.

2.2 Analyse après intégration (ydata_profiling)

Cette sous-partie répond à la **question 2 de l'étape 2** du sujet : analyser les données après intégration avec `ydata_profiling` (ou équivalent). Le rapport détaillé, rédigé en français, est joint en annexe à la fin de ce document.

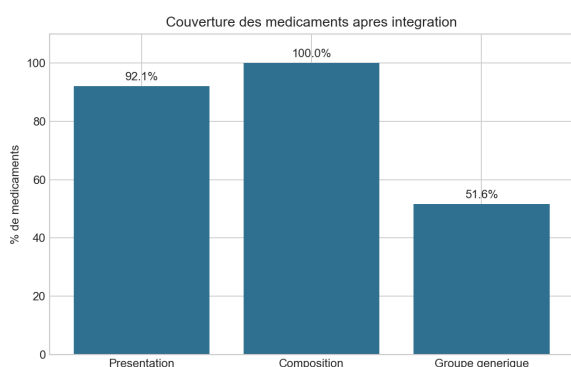


Figure 16 – Couverture des médicaments après intégration.

```
ydata_profiling - resume apres integration

Patients      17 498 lignes | ages 1..90 | genres equilibres
Medicaments   15 883 lignes | AWH active dominante
Presentations 29 921 lignes | 37% prix manquants
Prendre       echantillon | posologie 1..3, duree 1..30 OK
Compos       nature SA / P7
Appartenir    types 0/1/2/4 conformes

Couverture medicaments :
  presentations 92,1 %
  composition   100,0 %
  groupe gen.   51,6 %

Rapport HTML (français) : 07b_rapport_ydata_profiling.html
```

Figure 17 – Vue d'ensemble du rapport `ydata_profiling`.

On retient surtout que 92,1 % des médicaments ont une présentation, que tous ont une composition, mais que seulement 51,6 % appartiennent à un groupe générique. Environ 37 % des présentations n'ont pas de prix. Les âges des patients vont de 1 à 90 ans (moyenne 45,4), avec des genres globalement équilibrés. Les bornes imposées sur Prendre sont bien respectées.

3 Conclusion

En résumé, des fichiers parfois hétérogènes ont été transformés et chargés dans un schéma SQLite cohérent, complété des associations manquantes et de contraintes d'intégrité. La base peut ensuite être exploitée proprement par requêtes SQL et profiling.

A Annexe — Rapport ydata_profiling

Cette annexe correspond au livrable de la **question 2 de l'étape 2** du sujet : « *Utiliser pandas_profiling (ou tout autre bibliothèque Python type ydata_profiling) pour analyser les données après intégration* ».

Le document qui suit est le rapport généré par ydata_profiling sur la base SQLite cible, après chargement et application des contraintes. Il complète la synthèse présentée dans la section 2.2 et constitue la preuve détaillée de l'analyse post-intégration.

Les pages suivantes contiennent le rapport de profiling.



Vue d'ensemble

Vue d'ensemble

Alertes **1**

Reproduction

Statistiques du jeu de donnees

Nombre de variables	13
Nombre d'observations	20000
Cellules manquantes	0
Cellules manquantes (%)	0.0%
Lignes en double	0
Lignes en double (%)	0.0%
Taille totale en memoire	13.6 MiB
Taille moyenne d'un enregistrement	713.3 B

Types de variables

Texte	6
Numerique	3
Date/heure	1
Categoriel	3

Variables

Select Columns ▾

idfPatient

Texte

Distincts	11460
Distinctss (%)	57.3%
Manquants	0
Manquants (%)	0.0%
Taille memoire	1.1 MiB



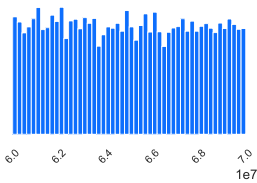
More details

idfMedic

Real number ($\mathbb{R}$)

Distincts	11394
Distinctss (%)	57.0%
Manquants	0
Manquants (%)	0.0%
Infinis	0
Infinis (%)	0.0%

Minimum	60002
Maximum	69999
Zeros	0
Zeros (%)	0.0%
Negative	0
Negative (%)	0.0%



Moyenne

649537

Taille
memoire

156.4 K

More details

datePrise

Date

Distincts

640

Minimum

2025-01-01 00:00:00

Distinctss (%)

3.2%

Maximum

2026-12-31 23:59:59

Manquants

0

Invalid
dates

0

Manquants (%)

0.0%

Invalid
dates (%)

0.0%

Taille
memoire

156.4 K

More details

posologie

Categoriel

Distincts

5

Distinctss (%)

< 0.1%

Manquants

0

Manquants (%)

0.0%

Taille memoire

976.7 KiB

1

4081

3

4016

4

4003

2

3973

5

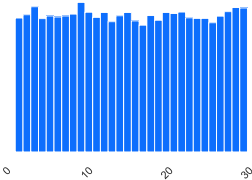
3927

More details

duree

Real number (ℝ)

Distincts	30	Minimum	1
Distinctss (%)	0.1%	Maximum	30
Manquants	0	Zeros	0
Manquants (%)	0.0%	Zeros (%)	0.0%
Infinis	0	Negative	0
Infinis (%)	0.0%	Negative (%)	0.0%
Moyenne	15.4996	Taille memoire	156.4

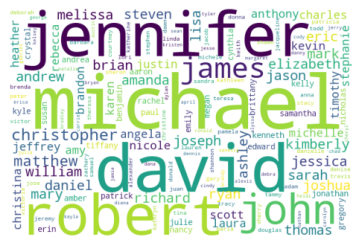


More details

prenom

Texte

Distincts	672
Distinctss (%)	3.4%
Manquants	0
Manquants (%)	0.0%
Taille memoire	1.0 MiB

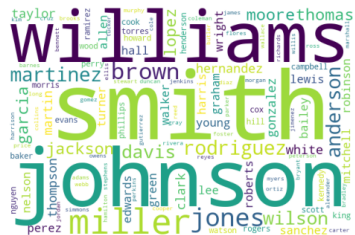


More details

nom

Texte

Distincts	993
Distinctss (%)	5.0%
Manquants	0
Manquants (%)	0.0%
Taille memoire	1.1 MiB

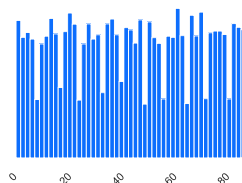


More details

age

Real number ($\mathbb{R}$)

Distincts	90	Minimum	1
Distinctss (%)	0.4%	Maximum	90
Manquants	0	Zeros	0
Manquants (%)	0.0%	Zeros (%)	0.0%
Infinis	0	Negative	0
Infinis (%)	0.0%	Negative (%)	0.0%



Moyenne 45.5419

**Taille
memoire** 156.4

More details

genre

Categoriel

Distincts 3

Distinctss (%) < 0.1%

Manquants 0

Manquants (%) 0.0%

Taille memoire 1.0 MiB

Other 6728

Female 6688

Male 6584

More details

mail

Texte

Distincts	11192
Distinctss (%)	56.0%
Manquants	0
Manquants (%)	0.0%
Taille memoire	1.4 MiB



More details

denomination

Texte

Distincts 11257

Distinctss (%)	56.3%
-----------------------	-------

Manquants	0
-----------	---

Manquants (%) 0.0%

Taille memoire	3.0 MiB
-----------------------	---------



More details

forme

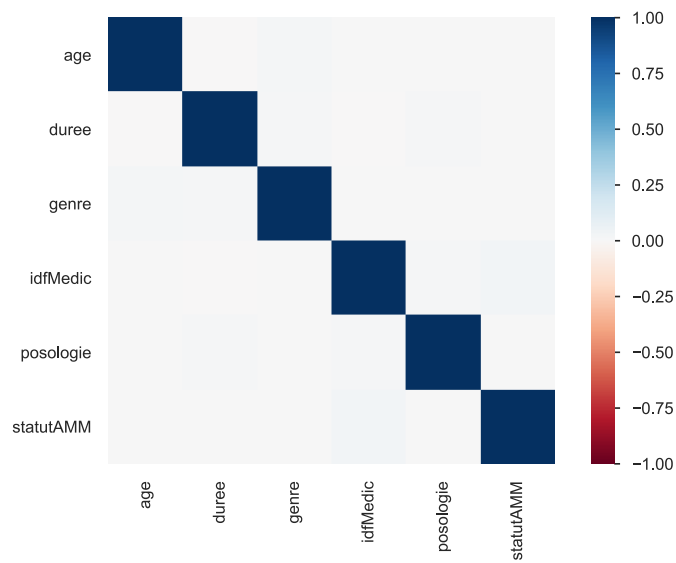
Texte

Distincts	338
Distinctss (%)	1.7%
Manquants	0
Manquants (%)	0.0%
Taille memoire	1.9 MiB

Auto

Heatmap

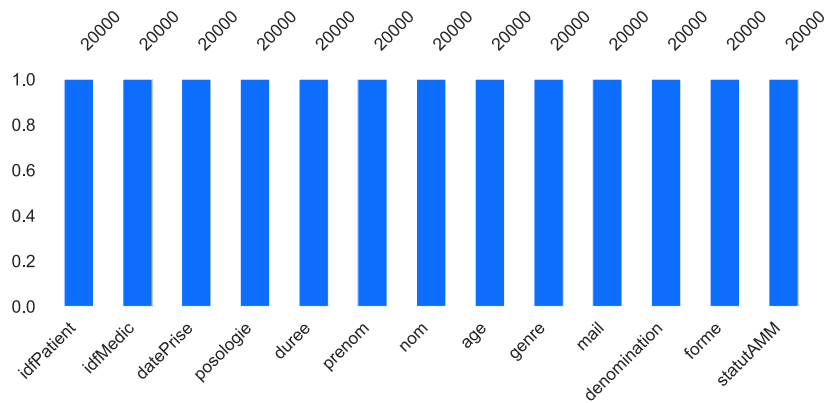
Table



Valeurs manquantes

Count

Matrix



A simple visualization of nullity by column.

Echantillon

First rows

Last rows

	idfPatient	idfMedic	datePrise	posologie	duree	pre
0	PID9747	60686803	2025-10-22	2	24	Mar
1	PID8098	67885195	2025-02-14	2	11	Mar
2	PID3082	64204250	2025-04-12	1	20	Step
3	PID15229	66738616	2026-02-18	2	27	Dar
4	PID2346	63531263	2025-04-07	3	8	Cha
5	PID6762	61606614	2025-11-18	2	8	Sara

	idfPatient	idfMedic	datePrise	posologie	duree	pre
6	PID12960	65979151	2025-09-26	5	14	Eliza
7	PID15443	67643377	2026-08-24	1	28	Dor
8	PID15082	60156662	2025-08-02	5	18	Jeff
9	PID14525	69211366	2025-01-23	1	13	Brat

Report generated by [YData](#).

